week_9\week9\src\main\java\week9\StatelessServer.java

```java
package week9;

import java.io.*;
import java.net.*;
/**
 * A simple server for the tic-tac-toe game that listens for client connections and allows
 * a multi client to play the game against a computer opponent.
 */
public class StatelessServer {

    /**
     * Main method to start the server and listen for client connections.
     * @param args
     * @throws IOException
     */
    public static void main(String[] args) throws IOException {
        try (ServerSocket serverSocket = new ServerSocket(9030)) {
            while(true){
                try (Socket socket = serverSocket.accept()){
                    System.out.println("Client connected: " + socket.getInetAddress());
                    handleClient(socket);
                }catch(IOException e){
                    System.out.println("Client disconnected: " + e.getMessage());
                }
                System.out.println("Client disconnected");

                /*  at the end of the try block,
                    the client socket will be closed automatically
                    This is equal to socket.close() in the finally block,
                    but more elegant and less error-prone.
                */
            }
        }catch(IOException e){
            System.out.println("Client disconnected: " + e.getMessage());

        }
        System.out.println("Server disconnected");
        /*  at the end of the try block,
            the server socket will be closed automatically
            This is equal to socket.close() in the finally block,
            but more elegant and less error-prone.
        */
    }

    /**
     * Handles client connection, check and then reply.
     * @param socket
     * @throws IOException
     */
    private static void handleClient(Socket socket) throws IOException {
```

```
51          // set up streams
52          BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
53          PrintStream out = new PrintStream(socket.getOutputStream(), true);
54
55          //set up computer
56          Computer computer = new Computer(Constants.COMPUTER_MARKER, "COMPUTER");
57
58          //Create board
59          Board board = new Board2D(System.out);
60
61          //Receive client string
62          board.loadString(in.readLine());
63
64          //Take player move, change from int → position
65
66          // Check if the client wants to quit
67          String clientInput = in.readLine();
68          if (clientInput.equals("quit")) {
69              System.out.println("Client requested to quit the game.");
70              return; // Exit the method to close the connection
71          }else{
72              // If not quitting, treat the input as a move
73              Integer playerMove = Integer.parseInt(clientInput);
74              Position playerPos = board.getCellPosition(playerMove);
75
76              //Check player move :')
77              if(board.isCellEmpty(playerPos)){
78                  //place player move :D
79                  board.placeMarker(playerPos, Constants.HUMAN_MARKER);
80              }else{
81                  out.println("Cell is occupied!");
82                  out.println(board.networkString());
83                  return;
84              }
85          }
86
87          //Check winners, continue if don't
88          String status = getGameStatus(board);
89
90          if(status.contains("Computer turns")){
91              board.placeMarker(computer.makeMove(board),
Constants.COMPUTER_MARKER);
92              status = getGameStatus(board);
93          }
94
95          out.println(status);
96          out.println(board.networkString());
97      }
98
99      // This is me again, get tired of writing long if else :') so I create a
helper
100     private static String getGameStatus(Board board){
101         char winner = board.checkWinner();
```

```
102         if (winner == Constants.HUMAN_MARKER) return "You wins!";
103         if (winner == Constants.COMPUTER_MARKER) return "Computer wins!";
104         if (board.isBoardFull()) return "Draw!";
105         return "Computer turns";
106     }
107 }
108
```